PRISMSPACE - INTELLIGENT MULTI-AGENT PLATFORM



Dept. of Computer Science & Engineering
College of Engineering, Karunagappally

Project Group 01

NOBIN SIJO	KNP23CS082
PRANAV P	KNP23CS085
PRASANTH P	KNP23CS086
SANKAR NARAYAN S	KNP23CS094

Project Coordinator & Guide
Dr. Leena Silvoster
Associate Professor
College of Engineering, Karunagappally

01. Introduction: PrismSpace Platform

PrismSpace is a local-first, multi-agent intelligence platform that converts natural-language user goals into dependency-aware **Directed Acyclic Graphs (DAGs)** for parallel sub-task execution. By pairing dynamic multi-model routing (Groq, OpenAI, NVIDIA NIM) with standardized tool access via the **Model Context Protocol (MCP)**, it coordinates specialized AI swarms to solve everyday and developer workflows—delivering safety-aligned verification and zero-latency local persistence (`PrismDB`) within a unified, reactive workspace.

02. Project Objectives

1. **Context-Aware Task Planning:** Convert everyday user goals (communication, file management, daily schedules) and technical requests into structured Directed Acyclic Graphs (DAGs) for organized, dependency-safe execution.

2. **Intelligent Model & Agent Routing:** Dynamically route each sub-task to the most suitable Large Language Model (LLM) backend and dispatch specialized, context-aware agents in parallel.

3. **Practical Tool Execution via MCP:** Connect agents to everyday productivity tools and development utilities (Gmail, Google Drive, Filesystem, Terminal CLI, GitHub) through the Model Context Protocol (MCP), accompanied by live Server-Sent Events (SSE) updates.

4. **Safety Verification & Local Persistence:** Validate outputs with human approval checkpoints and Odds Ratio Preference Optimization (ORPO), storing results reactively in local IndexedDB (Indexed Database: `PrismDB`).

03. PrismSpace Workflow (Figure 1.0)

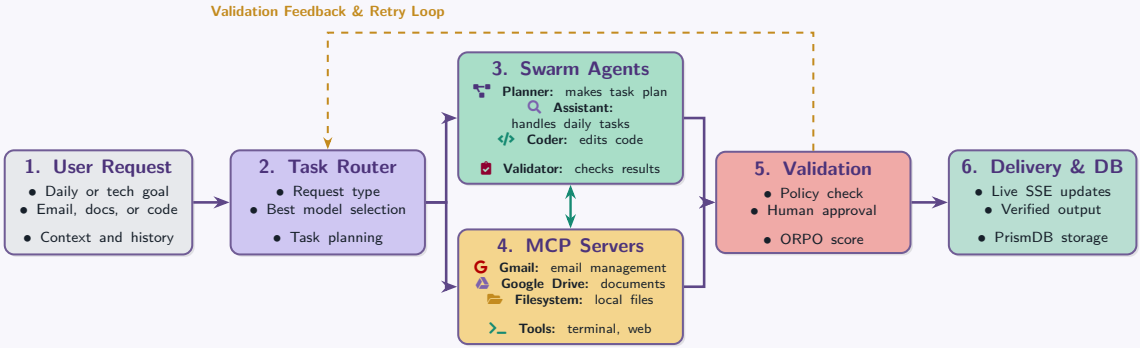


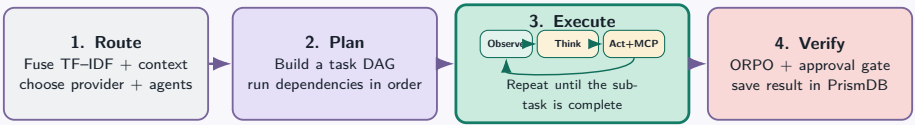
Figure 1.0: PrismSpace Workflow | **DAG** = Directed Acyclic Graph | **MCP** = Model Context Protocol | **ORPO** = Odds Ratio Preference Optimization | **SSE** = Server-Sent Events

04. System Design Principles

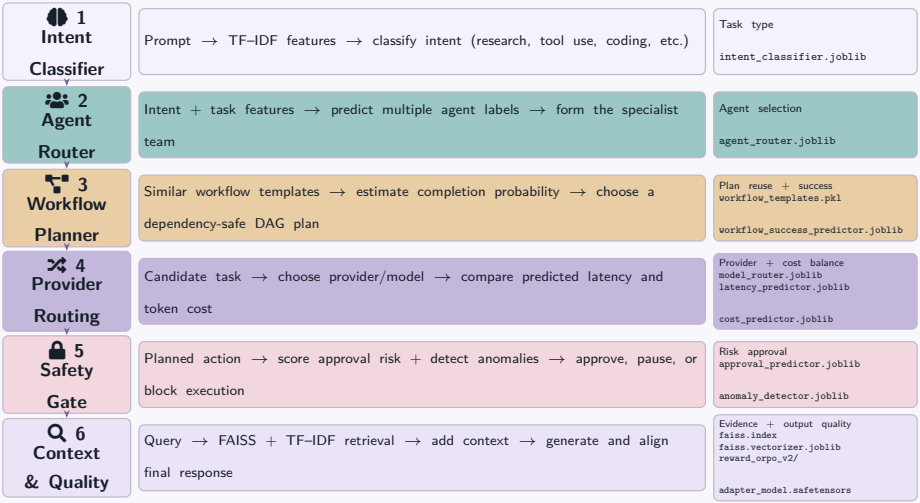
- **Layered Decoupling:** Separates the React client UI, FastAPI bridge, and foundation LLMs (each layer has one clear role).
- **Dynamic DAG Scheduling:** Runs dependent sub-tasks in parallel ($K \leq 5$; additional tasks are queued) for daily and technical workflows.
- **Standardized Protocol (MCP):** Uses open JSON-RPC tool contracts (a common way for agents and tools to communicate).
- **Human-in-the-Loop:** Sensitive operations require explicit user approval (a safety checkpoint before execution).
- **Local IndexedDB (`PrismDB`):** ACID browser storage (reliable local data) for sessions, tasks, tool history, and files.
- **Reactive Subscriptions:** Dexie `useLiveQuery()` updates the UI automatically when data changes.
- **Server-Sent Events (SSE):** Streams tokens and agent telemetry over persistent HTTP (live one-way updates).
- **Calibrated Preference Gating:** ORPO ranks and filters outputs to improve quality and safety.
- **Design Philosophy:** Context-aware, sandboxed, and observable multi-agent workspace (safe and traceable by design).

06. Proposed Methods

The proposed PrismSpace framework transforms complex natural-language requests into reliable, verified outcomes through a four-stage orchestration pipeline. It routes user intent via contextual embeddings to select optimal model providers and specialist agents, plans execution using a dependency-aware Directed Acyclic Graph (DAG), executes sub-tasks through iterative ReAct loops with standardized MCP tools, and validates results using ORPO preference alignment and approval gates.



05. Model Responsibilities & Inference Flow



10. Unified Dataset Preparation & Multi-Model Training Pipeline

1. Data Sources

24 training sets covering:

- Agent traces: timings, tool calls, and task trajectories.
- Tool calling: API, parallel-call, and coding benchmarks.
- Retrieval data: search corpora, schemas, and dialogues.
- Safety and preference data: harm labels and ranked responses.

2. Curate & Train

- `dataset_loader.py`: combines JSON, CSV, Parquet, PKL, and Markdown.
- Normalizes prompts, actions, timestamps, and labels into one schema.
- Supervised preparation creates approval, success, provider, and reward splits.
- `train.py`: uses TF-IDF/FAISS retrieval, K-Means workflow clustering, and scikit-learn predictors.
- ORPO fine-tunes the reward model on Qwen2.5 for preference alignment.

3. Artifacts & Evaluate

- **11 production artifacts**.
- Intent classifier and multi-label router classify tasks and select specialists.
- K-Means clusters workflows; predictors estimate success, latency, and cost.
- Approval classifier and Isolation Forest screen risky or unusual actions.
- TF-IDF/FAISS retrieve context; ORPO aligns responses.
- Holdout tests check F1, precision, recall, calibration, GAIA, and SWE-bench.



08. How the System Works: From Request to Result

Stage 1: Ingestion & Features

- **Capture request:** Read the user's goal.
- **Clean text:** Remove noise and missing values.
- **Extract features:** Convert the request into useful signals.

Stage 2: Intelligence & DAG

- **Classify intent:** Decide what kind of task it is.
- **Choose agents and models:** Match each sub-task to the right worker.
- **Build the DAG:** Arrange tasks in dependency order and check safety.

Stage 3: Execution & Storage

- **Execute:** Run ready tasks with MCP tools.
- **Stream updates:** Show progress and logs live.
- **Check and save:** Verify the result and store the run.

How a DAG is created

1. User request
"Plan a report"

→ **2. Split into sub-tasks**

→ **3. Add dependencies**

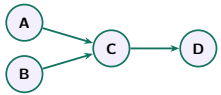
→ **4. Create DAG $\mathcal{G}(V, E)$**

→ **5. Run ready tasks in parallel**

Dependency-Aware Topological Execution:

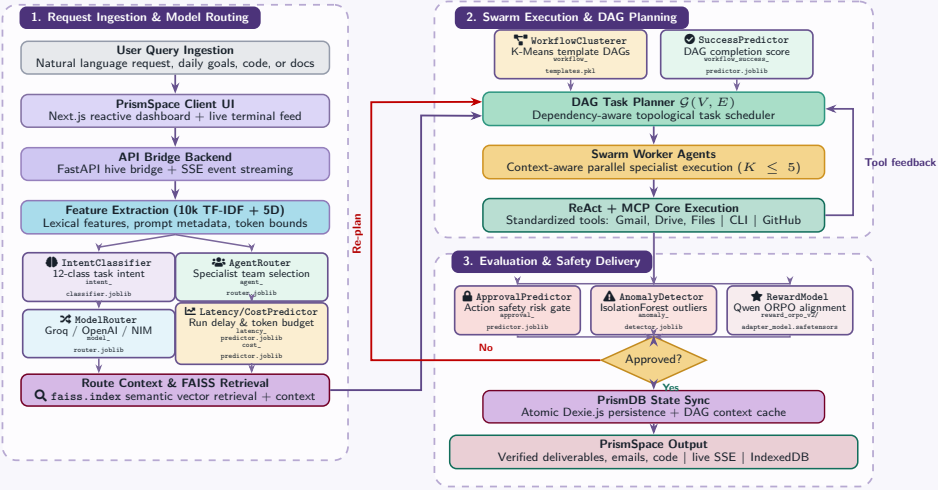
Independent sub-tasks execute concurrently ($A \parallel B$). Downstream tasks wait until prerequisite dependencies resolve before starting.

Example:



A/B run together → C waits for both → D finishes

09. PrismSpace System Architecture & Multi-Model Swarm Workflow (Figure 1.2)



09. Browser Database Schema: PrismDB (Figure 1.3)

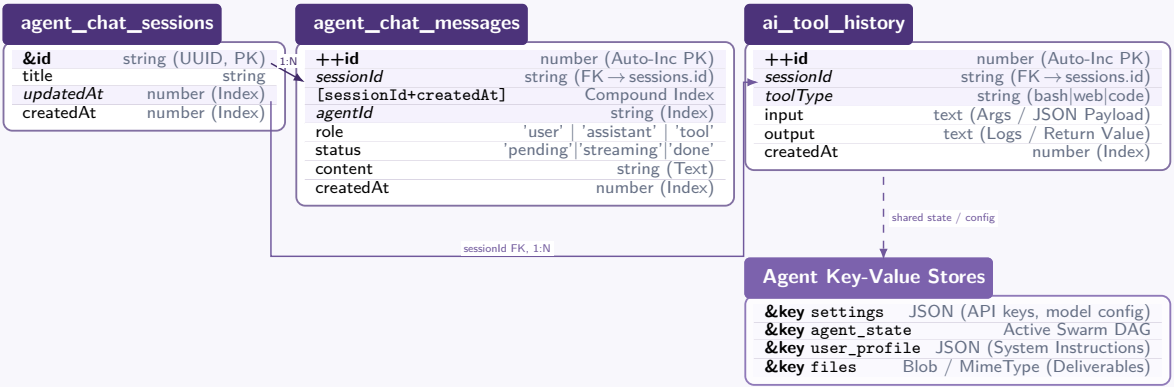


Figure 1.3: PrismDB — AI-Agent Browser IndexedDB Schema (Dexie.js v3) | **PK** = Primary Key **FK** = Foreign Key **Index** = B-tree index | FK / Indexed row Key-Value store

`sessions.id` shared FK | `sessions.id` → `messages.sessionId` | `sessions.id` → `tool_history.sessionId` | `useLiveQuery()` · ACID transactions · Dexie.js v3

Algorithm: Step 1 – Understand the User Prompt

Step 1: Prompt Understanding

- **Goal & Intent Identification:**
  - Reads natural-language request and extracts the primary user objective.
  - Incorporates previous conversation turns and project context from PrismDB.
- **Constraint & Context Formalization:**
  - Extracts operational constraints, local files, deadlines, and expected deliverables.
  - **Task Tuple Representation:** Formalized as $T = (G, C, X, R, O)$ where:
    - G: User goal, C: Constraints, X: Context, R: Tool resources, O: Expected output.
- **Safety Screening & Router Hand-off:**
  - Detects if request requires external MCP tools, human approval, or extra clarification.
  - **Methods Used:**
    - **NLU parsing:** Extracts intent, entities, and action requirements from request syntax.
    - **Semantic vector retrieval:** Matches the request to relevant context by meaning.
    - **Lexical feature extraction:** Scores keywords, constraints, and deadlines for downstream routing.

Algorithm: Step 2 – Classify the Request

Step 2: Intent and Requirement Analysis

- **Multi-Class Intent Categorization:**
  - Maps request into 12 distinct task classes (everyday, document, research, code, mixed).
  - **Optimal Class Selection:** Formulated via maximum posterior probability:

$$c^* = \arg\max_{c \in \mathcal{C}} P(c \mid x)$$

    where x represents the extracted 10k TF-IDF lexical and token prompt features.

- **Capability & Action Boundary Profiling:**
  - Identifies required capabilities: file I/O, search, synthesis, computation, or communication.
  - Classifies action types into safe read-only, state-modifying, or sensitive operations.

- **Risk Assessment & Safety Screening:**
  - Evaluates complexity, execution urgency, and initial risk levels.
  - **Methods Used:** Supervised classification predicts the task class; keyword scoring extracts capabilities, constraints, and urgency; rule-based gating flags sensitive or state-changing actions.

Algorithm: Step 3 – Plan and Route the Task

Step 3: Task Decomposition and Model Routing

- **DAG Workflow Construction:**
  - Decomposes compound request into modular sub-tasks with dependency constraints.
  - Generates Directed Acyclic Graph $\mathcal{G}(V, E)$ via topological sorting for parallel execution.
- **Multi-Provider Cost/Latency Optimization:**
  - Routes sub-tasks to Groq (ultra-low latency), OpenAI (reasoning), or NVIDIA NIM.
  - **Objective Function:** Minimizes Pareto cost and latency subject to quality threshold τ_Q:

$$m^* = \arg\min_m (\lambda_c C_m + \lambda_l L_m) \quad \text{s.t.} \quad Q_m \geq \tau_Q$$

- **Concurrency Dispatch:**
  - Identifies zero-in-degree nodes and dispatches non-dependent tasks in parallel.
  - **Methods Used:** Topological scheduling orders dependency-safe tasks; multi-criteria routing weighs quality, cost, and latency; Pareto optimization selects efficient provider/model trade-offs.

Step 4: Swarm Agent Selection

- **Specialist Agent Allocation:**
  - Matches sub-tasks to designated worker roles: *Planner, Assistant, Researcher, Coder, Reviewer*.
  - **Agent Assignment Function:** Evaluates capability alignment across candidate agent pool $\mathcal{A}$:

$$a^* = \arg\max_{a \in \mathcal{A}} \mathrm{Match}(a, \text{Requirements})$$

- **Specialist Role Responsibilities:**
  - *Assistant Agent*: Handles communications, documents, and calendar scheduling.
  - *Researcher & Coder*: Synthesize multi-source data and generate/verify code artifacts.
  - *Reviewer Agent*: Validates intermediate node outputs and flags errors.
- **Swarm Collaboration Mechanics:**
  - Binds agents with shared memory context and synchronized DAG state.
  - **Methods Used:** Role-based routing assigns specialist work; capability matching aligns skills with sub-task requirements; parallel swarm dispatch runs independent assignments concurrently.

Algorithm: Step 5 – Execute with MCP Tools

Step 5: ReAct Reasoning and MCP Tool Use

- **ReAct Decision Cycle:**
  - Executes continuous reasoning cycle: Observation → Thought → Action → Observation.
  - **Action Value Optimization:** Selects maximum expected-utility action for state s_t:

$$a_t = \arg\max_a Q(s_t, a)$$

- **Standardized MCP Server Invocation:**
  - Interacts with local tools via JSON-RPC: Gmail, Drive, Filesystem, Terminal CLI, GitHub.
  - Ingests tool outputs back into agent context for downstream reasoning iterations.
- **Human-in-the-Loop Safeguards:**
  - Pauses execution before state-mutating or high-risk actions pending explicit human approval.
  - **Methods Used:** ReAct prompting alternates reasoning and tool actions; MCP schemas standardize JSON-RPC calls and outputs; sandboxed execution isolates tools and pauses sensitive operations for approval.

Algorithm: Step 6 – Share Context and Memory

Step 6: Context Synchronization

- **Inter-Agent State Propagation:**
  - Transfers completed sub-task outputs to dependent downstream agents along DAG edges.
  - Preserves global user intent, constraints, and decision logs across all transitions.
- **Dynamic Context Compaction:**
  - Deduplicates intermediate chatter and prunes obsolete context to stay within token limits.
  - **Context Update Equation:** Formalized state update with compaction:

$$C_{t+1} = \text{Compress}\big(C_t \cup \Delta C_t \setminus D_t\big)$$

    where ΔC_t represents new tool outputs and D_t denotes evicted obsolete tokens.
- **Reactive Local Storage:**
  - Writes state changes atomically to browser IndexedDB via Dexie.js (`PrismDB`).
  - **Methods Used:** Shared-memory caching propagates state across agents; token pruning removes redundant or stale context; atomic persistence preserves recoverable local updates.

Step 7: Verification and Safety

- **Multi-Criteria Output Auditing:**
  - Reviews deliverables against original goal, formatting constraints, and safety guidelines.
  - Evaluates action outputs with IsolationForest anomaly detection and WildGuardMix checks.
- **Composite Safety & Alignment Gating:**
  - Scores outputs against human preference via Qwen ORPO alignment reward model.
  - **Approval Gating Condition:** Only permits completion if composite score passes threshold:

$$S = \sum_i w_i \, q_i \geq \tau_{\text{safety}}$$

    where q_i evaluates correctness, completeness, and safety with relative weights w_i.
- **Corrective Re-plan Loop:**
  - If verification fails, routes task back to the DAG Planner with feedback for targeted retry.
  - **Methods Used:** ORPO preference scoring ranks quality and alignment; anomaly isolation localizes unsafe or faulty outputs; automated feedback loops trigger targeted retry.

Algorithm: Step 8 – Deliver and Monitor

Step 8: Verified Delivery and Monitoring

- **Result Synthesis & Live Streaming:**
  - Assembles verified sub-task outputs into final deliverables (emails, code, reports).
  - Streams real-time progress, intermediate logs, and final answers to UI via Server-Sent Events (SSE).
- **End-to-End Quality Assessment:**
  - Evaluates holistic delivery performance across execution metrics.
  - **Delivery Score Formulation:** Multiplicative composite metric:

$$D = \text{Correctness} \times \text{Completeness} \times \text{Timeliness}$$

- **Persistence & Continuous Learning:**
  - Persists conversation, DAG nodes, and tool outputs into Dexie.js (`PrismDB`).
  - Records latency, costs, and approval patterns for offline model retraining.
  - **Methods Used:** SSE streaming exposes live progress; reactive Dexie persistence stores execution state; telemetry collection records latency, cost, and approval signals for learning.

Thank You!

Questions & Discussion

Nobin Sijo | Pranav P | Prasanth P | Sankar Narayan S

Department of Computer Science & Engineering
College of Engineering, Karunagappally